

指针与抽象

结构体、指针与STL容器 · 10题 · C++ & Python 双语对照

NQ096 替换空格 AcWing 16

请实现一个函数，把字符串中的每个空格替换成%20。

输入 输入一个字符串。数据范围：0 ≤ 输入字符串的长度 ≤ 1000

输出 输出替换后的字符串。

来源 AcWing 16

输入
We are happy.

输出
We%20are%20happy.

提示 注意输出格式。

C++

```
class Solution {
public:
    string replaceSpaces(string &str) {
        string res;
        for (auto c : str)
            if (c == ' ') res += "%20";
            else res += c;
        return res;
    }
};
```

Python

```
# NQ016 十六进制加法
import sys
for line in sys.stdin:
    line=line.strip()
    if not line:continue
    parts=line.split()
    if len(parts)!=2:continue
    a=int(parts[0],16);b=int(parts
[1],16) # 十六进制转整数
    res=a+b
    if res<0:
        print('-'+hex(-res)[2:].upper
()) # 负数处理
    else:
        print(hex(res)[2:].upper())
# 大写十六进制输出
```

NQ097 从尾到头打印链表 AcWing 17

输入一个链表的头结点，按照从尾到头的顺序返回节点的值。返回的结果用数组存储。

输入 链表节点值序列，以-1结束。

输出 从尾到头的节点值，每行一个。

来源 AcWing 17

输入
1 2 3 -1

输出

3
2
1

提示 数据范围: 链表长度 ≤ 1000

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    vector<int> printListReversingly(ListNode* head) {
        vector<int> res;
        if (!head) return res;
        auto p = head;
        while(p) {
            res.push_back(p->val);
            p=p->next;
        }
        reverse(res.begin(), res.end());
        return res;
    }
};
```

Python

```
# NQ017 计算绩点
q=int(input())
tc=0.0;tg=0.0
for _ in range(q):
    s,c=map(float,input().split())
    # 根据成绩计算绩点
    if s>=90:g=4.0
    elif s>=85:g=3.7
    elif s>=81:g=3.3
    elif s>=78:g=3.0
    elif s>=75:g=2.7
    elif s>=72:g=2.3
    elif s>=68:g=2.0
    elif s>=64:g=1.7
    elif s>=60:g=1.0
    else:g=0.0
    tc+=c;tg+=g*c # 累加学分和加权绩点
print('{0:.4f}'.format(tg/tc)) # 输出
加权平均绩点
```

NQ098 用两个栈实现队列 AcWing 20

请用栈实现一个队列，支持如下四种操作：push(x) – 将元素 x 插到队尾；pop() – 将队首的元素弹出，并返回该元素；peek() – 返回队首元素；empty() – 返回队列是否为空。

输入

输入数据保证合法，例如，在队列为空时，不会进行 pop 或者 peek 等操作。数据范围：每组数据操作命令数量[0, 100]。

输出

输出每个操作的结果。

来源

AcWing 20

输入

```
MyQueue queue = new MyQueue();
queue.push(1);
queue.push(2);
queue.peek();
queue.pop();
queue.empty();
```

输出

```
1
1
false
```

提示 注意：你只能使用栈的标准操作：push to top, peek/pop from top, size和is empty；如果你选择的编程语言没有栈的标准库，你可以使用list或者deque等模拟栈的操作；输入数据保证合法，例如，在队列为空

时，不会进行pop或者peek等操作；

C++

```
class MyQueue {
public:
    stack<int> stk, cache;
    /** Initialize your data structure here. */
    MyQueue() {}

    /** Push element x to the back of queue. */
    void push(int x) {
        stk.push(x);
    }

    void copy(stack<int>& a, stack<int>& b){
        while (a.size()) {
            b.push(a.top());
            a.pop();
        }
    }

    /** Removes the element from in front of queue and returns that element. */
    int pop() {
        copy(stk,cache);//拷贝到cache数组
        int res = cache.top();
        cache.pop();
        copy(cache,stk);
        return res;
    }

    /** Get the front element. */
    int peek() {
        copy(stk,cache);//拷贝到cache数组
        int res = cache.top();
        copy(cache,stk);
        return res;
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return stk.empty();
    }
};

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * bool param_4 = obj.empty();
 */
```

Python

```
import sys

def main():
    try:
        input = sys.stdin.read
        data = input().split()
        if not data: return

        iterator = iter(data)
        t_str = next(iterator, None)
        if not t_str: return
        t = int(t_str)

        for _ in range(t):
            h = int(next(iterator))
            m = int(next(iterator))
            s = int(next(iterator))

            # 分针角度: 每分钟6度
            m_ang = (m + s / 60.0) * 6.0

            # 时针角度: 每小时30度
            h_ang = (h % 12 + (m + s / 60.0) / 60.0) * 30.0

            diff = abs(h_ang - m_ang)
            if diff > 180:
                diff = 360 - diff

            print(int(diff))
    except StopIteration:
        pass

if __name__ == "__main__":
    main()
```

NQ099 斐波那契数列 AcWing 21

输入一个整数n，要求斐波那契数列的第n项。假定从0开始，第0项为0。

输入 输入一个整数n。数据范围： $0 \leq n \leq 39$

输出 返回斐波那契数列的第n项。

来源 AcWing 21

输入

5

输出

5

提示 注意输出格式。

C++

```
class Solution {
public:
    int Fibonacci(int n) {
        if (n <= 1) return n;
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
};
```

Python

```
# NQ021 废强重建
import sys
first=True
while True:
    line=sys.stdin.readline()
    if not line:break
    n=int(line.strip())
    if n==0:break
    h=[]
    while len(h)<n:
        hl=sys.stdin.readline()
        if not hl:break
        h.extend(map(int,hl.strip().split()))
    total=sum(h)
    avg=total//n # 计算平均值 (向下取整)
    need=sum(avg-ht for ht in h if ht < avg) # 统计需要的砖块数
    if not first:print()
    print(need)
    first=False
```

NQ100 反转链表 AcWing 35

定义一个函数，输入一个链表的头结点，反转该链表并输出反转后链表的头结点。

输入 输入一个链表的头结点。数据范围：链表长度[0, 30]。

输出 输出反转后的链表的头结点。

来源 AcWing 35

输入

1->2->3->4->5->NULL

输出

5->4->3->2->1->NULL

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next) return head;

        auto p = head, q = p->next;
        while (q)
        {
            auto o = q->next;
            q->next = p;
            p = q, q = o;
        }
        head->next = NULL;

        return p;
    }
};

```

Python

```

# NQ035 二进制字符串奇偶位互换
# 将二进制字符串的相邻两位（奇偶位）互换位置
def main():
    test_case_count = int(input()) # 读取测试用例数量

    for _ in range(test_case_count):
        # 遍历每个测试用例
        binary_str = input().strip()
        # 读取二进制字符串
        chars = list(binary_str) # 转换为字符列表以便修改

        # 每两位互换位置（奇偶位互换）
        # 遍历步长为2，每次处理i和i+1位置的字符
        for i in range(0, len(chars) - 1, 2):
            chars[i], chars[i + 1] = chars[i + 1], chars[i]

        print(''.join(chars)) # 转换回字符串并输出

if __name__ == "__main__":
    main()

```

NQ101 合并两个排序的链表 AcWing 36

输入两个递增排序的链表，合并这两个链表并使新链表中的结点仍然是按照递增排序的。

输入 输入两个链表。数据范围：链表长度[0, 500]

输出 输出合并后的链表。

来源 AcWing 36

输入

1->3->5
2->4->5

输出

1->2->3->4->5->5

提示 注意输出格式。

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* merge(ListNode* l1, ListNode* l2) {
        auto dummy = new ListNode(-1), tail = dummy;
        while (l1 && l2)
            if (l1->val < l2->val)
            {
                tail = tail->next = l1;
                l1 = l1->next;
            }
            else
            {
                tail = tail->next = l2;
                l2 = l2->next;
            }

            if (l1) tail->next = l1;
            if (l2) tail->next = l2;

            return dummy->next;
    }
};

```

Python

```

# NQ036 合并字符串初阶
def main():
    import sys
    input_lines = [line.rstrip('\n')
    for line in sys.stdin]
    ptr = 0
    t = int(input_lines[ptr]) # 读入
    测试用例数量
    ptr += 1

    for _ in range(t):
        a = input_lines[ptr] # 读入第
        一行
        ptr += 1
        b = input_lines[ptr] # 读入第
        二行
        ptr += 1

        p = len(a) // 2 # 取中点
        # 合并字符串: a的前p个字符 + b +
        a的后部分
        print(a[:p] + b + a[p:])

if __name__ == "__main__":
    main()

```

NQ102 三元组排序 AcWing 862

给定N个三元组(x,y,z)，每个三元组包含一个整数x、一个整数y和一个字符串z。按照x从小到大的顺序输出所有三元组。数据保证不同三元组的x值互不相同。

输入 第一行包含整数N。接下来N行，每行包含一个整数x、一个整数y和一个由小写字母组成的字符串z。

输出 共N行，按照x从小到大的顺序，每行输出一个三元组。

来源 AcWing 862

输入	输出
3	1 2 abc
1 2 abc	2 1 ghi
3 4 def	3 4 def
2 1 ghi	

提示 数据范围: $1 \leq N \leq 10000$, $1 \leq x, y \leq 10^5$, 字符串总长度不超过 10^5

C++

```

#include <iostream>
#include <algorithm>

using namespace std;

const int N = 10010;

struct data {
    int x;
    double y;
    string z;
    bool operator< (const data & t) const
    {
        return x < t.x;
    }
} a[N];

int main() {
    int n;
    cin>>n;
    for (int i=0; i< n; i++) cin>> a[i].x >> a[i].y >> a[i].z;
    //结构体排序
    sort(a, a+n);
    //输出结果
    for (int i = 0; i < n; i++)
        printf("%d %.2lf %s\n", a[i].x, a[i].y, a[i].z.c_str());
    return 0;
}

```

Python

Python solution pending

NQ103 绝对值 AcWing 810

输入一个整数 x ，请你编写一个函数 `int abs(int x)`，输出 x 的绝对值。

输入 共一行，包含一个整数 x 。

输出 共一行，包含 x 的绝对值。

来源 AcWing 810

输入

-5

输出

5

提示 数据范围: $-100 \leq x \leq 100$

C++

```
#include <iostream>

using namespace std;

int abs(int x)
{
    if (x > 0) return x;
    return -x;
}

int main()
{
    int x;
    cin >> x;
    cout << abs(x) << endl;

    return 0;
}
```

Python

Python solution pending

NQ104 复制数组 AcWing 814

要编写函数 `void copy(int a[], int b[], int size)` 将 `a` 数组中的前 `size` 个数复制到 `b` 数组中并输出 `b` 数组。

输入 第一行 `n, m, size` 分别表示 `a` 数组长度、`b` 数组长度和要复制的个数；第二行 `n` 个整数为数组 `a`；第三行 `m` 个整数为数组 `b`。数据范围： $1 \leq n \leq m \leq 100$, $1 \leq size \leq n$

输出 一行 `m` 个整数表示复制后的数组 `b`。

来源 AcWing 814

输入

```
3 5 2
1 2 3
4 5 6 7 8
```

输出

```
1 2 6 7 8
```

提示 注意输出格式。来源：语法题

C++

```

#include <iostream>
#include <cstring>

using namespace std;

const int N = 110;

void copy(int a[], int b[], int size)
{
    memcpy(b, a, size * 4);
}

int main()
{
    int a[N], b[N];
    int n, m, size;
    cin >> n >> m >> size;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    copy(a, b, size);

    for (int i = 0; i < m; i++) cout << b[i] << ' ';
    cout << endl;

    return 0;
}

```

Python

Python solution pending

NQ105 数组翻转 AcWing 816

编写函数 `void reverse(int a[], int size)` 实现将数组 `a` 中的前 `size` 个数翻转并输出翻转后的数组 `a`。

输入 第一行 `n` 和 `size` 分别表示数组长度和要翻转的个数；第二行 `n` 个整数为数组 `a`。数据范围： $1 \leq \text{size} \leq n \leq 1000$, $1 \leq a[i] \leq 1000$

输出 一行 `n` 个整数表示翻转后的数组 `a`。

来源 AcWing 816

输入

```
5 3
1 2 3 4 5
```

输出

```
3 2 1 4 5
```

提示 注意输出格式。来源：语法题

C++

```
#include <iostream>

using namespace std;

void reverse(int a[], int size)
{
    for (int i = 0, j = size - 1; i < j; i ++, j -- )
        swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, size;

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];
    reverse(a, size);

    for (int i = 0; i < n; i ++ ) cout << a[i] << ' ';

    return 0;
}
```

Python

Python solution pending